

NLP FOUNDATIONS

One-Hot Encoding · CBOW & Skip-gram · TF-IDF · Bag of Words · GloVe · Word Embeddings

1. One-Hot Encoding

One-hot encoding represents each word as a binary vector with a single 1 in the position corresponding to that word in the vocabulary, and 0s elsewhere.

Q1. A vocabulary is built from four fruit names: ["apple", "banana", "cherry", "date"]. Write the one-hot encoded vector for the word "cherry".

Answer: Assign each word a fixed position in the vocabulary:

apple → position 1, banana → position 2, cherry → position 3, date → position 4.

"cherry" is the 3rd word, so its one-hot vector is:

[0, 0, 1, 0]

Q2. An e-commerce catalog one-hot encodes 5 product categories: Electronics, Clothing, Grocery, Books, Toys. Six months later, a new category "Furniture" is introduced. What problem does this create for the existing one-hot encoding, and how would you handle it?

Answer: One-hot encoding uses a fixed-size vector equal to the vocabulary size (5 categories → 5-dimensional vectors). Adding "Furniture" changes the vocabulary size to 6, so:

- Every existing vector must be re-encoded to length 6 (a new 0 added for the new category).
- Any downstream model (e.g. a classifier) trained on 5-dimensional input must be retrained, since its input shape has changed.

Fix: either retrain the encoding and model whenever the category set changes, or reserve extra "unused" slots in advance, or switch to an encoding (like embeddings) that doesn't require a fixed, closed vocabulary.

Q3. In a 10,000-word vocabulary, the words "cat" and "dog" are one-hot encoded. What is the cosine similarity between their two vectors, and what limitation of one-hot encoding does this reveal?

Answer: Every one-hot vector has exactly one 1 in a different position from every other word's vector, so any two distinct one-hot vectors are orthogonal.

$$\text{Cosine similarity} = (\mathbf{v}_1 \cdot \mathbf{v}_2) / (||\mathbf{v}_1|| ||\mathbf{v}_2||) = 0 / (1 \times 1) = 0$$

This reveals that one-hot encoding treats every pair of words as equally unrelated — it cannot capture that "cat" and "dog" are semantically closer to each other than "cat" and "airplane" are. This is the key motivation for word embeddings.

2. CBOW and Skip-gram

CBOW (Continuous Bag of Words) predicts a target word from its surrounding context; Skip-gram predicts the surrounding context words from a single target word.

Q1. A company trains a Word2Vec model on a large news corpus that contains many rare technical terms alongside common everyday words. Which architecture — CBOW or Skip-gram — would you recommend, and why?

Answer: Recommendation: Skip-gram.

Reasoning: Skip-gram generates one training pair per (target, context word), so a rare word that appears only a few times still produces several training examples — one for each context word around each of its occurrences. CBOW averages all context words into a single input per target, which under-represents rare words. Skip-gram is therefore better at learning good vectors for infrequent, specialized terms.

Q2. Consider the sentence: "The stock market crashed today" with a window size of 2, and target word "crashed". List the single CBOW training instance (context words → target) for this window.

Answer: Word positions: The(0) stock(1) market(2) crashed(3) today(4)

Window size 2 around "crashed" (position 3) covers positions 1–5: stock, market, today (position 5 doesn't exist, so only 3 context words are available).

CBOW training instance:

Input (context): {stock, market, today} → Output (target): crashed

Q3. A startup has only a small, specialized medical corpus to train word vectors on. Which architecture typically converges better with limited data, and why?

Answer: Recommendation: Skip-gram.

Reasoning: With limited data, Skip-gram makes more efficient use of each sentence because it creates a separate (target, context) training pair for every context word, effectively multiplying the number of training examples generated from the same corpus. CBOW instead compresses the whole context into one averaged input per target word, producing fewer effective training signals — a disadvantage when data is scarce.

3. TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF scores a word by how often it appears in a document (TF), discounted by how common it is across all documents (IDF) — so common words score low and distinctive words score high.

Q1. Given a 3-document corpus — Doc1: "the cat sat on the mat", Doc2: "the dog sat on the log", Doc3: "cats and dogs are pets" — compute the TF-IDF score of the word "sat" in Doc1. (Use log base 10 for IDF.)

Answer: $TF(sat, Doc1) = (\text{occurrences of "sat" in Doc1}) / (\text{total words in Doc1}) = 1 / 6 \approx 0.167$

"sat" appears in Doc1 and Doc2, so document frequency $df = 2$, and $N = 3$ documents.

$IDF(sat) = \log_{10}(N / df) = \log_{10}(3 / 2) = \log_{10}(1.5) \approx 0.176$

$TF-IDF(sat, Doc1) = TF \times IDF \approx 0.167 \times 0.176 \approx 0.0294$

Q2. A search engine ranks documents for the query "python programming". Document A repeats the word "programming" 20 times but never mentions "python". Document B mentions both "python" and "programming" once each. Which document is likely to score higher, and why?

Answer: Document B is likely to score higher for this two-word query.

TF-IDF scores are computed per query term and then combined (e.g. summed) across all query terms. Document A racks up a large TF-IDF contribution from "programming" alone but contributes zero for "python" — it only partially matches the query. Document B

contributes a (smaller) TF-IDF score for each of the two query terms, so its combined score covers the full query intent rather than just one keyword repeated many times.

Q3. In a 1,000-document corpus, the word "the" appears in 950 documents, while "blockchain" appears in only 5. Compare their IDF values and explain why this matters when selecting features for a text classifier.

Answer: $\text{IDF}(\text{"the"}) = \log_{10}(1000/950) = \log_{10}(1.053) \approx 0.022$ (very low)

$\text{IDF}(\text{"blockchain"}) = \log_{10}(1000/5) = \log_{10}(200) \approx 2.301$ (much higher)

"blockchain" gets a far higher IDF because it appears in very few documents, making it a distinctive, informative feature. "the" appears almost everywhere, so its IDF is nearly zero — it contributes almost no discriminative power. TF-IDF therefore automatically down-weights near-universal stopwords and up-weights rare, topic-specific words, which is exactly the behavior wanted when selecting features for classification.

4. Bag of Words (BOW)

Bag of Words represents a document as a vector of word counts (or presence/absence) over a fixed vocabulary, ignoring grammar and word order.

Q1. Using the vocabulary [I, love, NLP, machine, learning], construct the BOW vectors for the sentences "I love NLP" and "I love machine learning".

Answer: Vocabulary order: [I, love, NLP, machine, learning]

"I love NLP" $\rightarrow [1, 1, 1, 0, 0]$

"I love machine learning" $\rightarrow [1, 1, 0, 1, 1]$

Each position counts occurrences of that vocabulary word in the sentence.

Q2. A spam classifier uses BOW features built from single words. Compare how BOW represents "not good" versus "good", and explain the weakness this exposes. Suggest one fix.

Answer: BOW treats each word independently, so "not good" is represented simply as the presence of both "not" and "good" as two separate features — the same two features would also appear (along with others) in a sentence like "good, not bad". BOW has no mechanism

to capture that "not" negates the word right after it, so it cannot distinguish positive from negated-positive sentiment from single-word features alone.

Fix: use bigrams/n-grams (so "not_good" becomes its own feature distinct from "good"), or replace BOW with word embeddings / contextual models that encode word order and local context.

Q3. Two reviews — "The service was great, not bad at all" and "The service was not great, quite bad" — express opposite sentiment. Explain why a simple BOW (word-presence) representation could produce very similar or identical vectors for both.

Answer: Both sentences use almost the exact same set of words: {the, service, was, great, not, bad, at, all, quite}. Since BOW only records which words are present (or how many times), not their order or which word each "not" attaches to, the two reviews map to nearly identical vectors — even though "not bad" (positive) and "not great" (negative) mean opposite things. This demonstrates BOW's core limitation: it discards word order and syntactic structure, so sentiment or meaning that depends on sequence is lost.

5. GloVe (Global Vectors for Word Representation)

GloVe learns word vectors by factorizing a global word–word co-occurrence matrix built from the entire corpus, rather than predicting words from local context windows.

Q1. A data scientist has access to global word co-occurrence statistics for a huge corpus. How does GloVe use this differently from Skip-gram, which trains using local context windows?

Answer: GloVe first builds a global co-occurrence matrix that counts, across the entire corpus, how often every pair of words appears near each other. It then learns word vectors by factorizing this matrix so that the dot product of two word vectors approximates the logarithm of their co-occurrence count.

Skip-gram, by contrast, never builds a global matrix — it slides a local window across the text and trains a predictive model (target word → context words) one window at a time, learning vectors indirectly through many small local predictions.

In short: GloVe learns from aggregated global statistics directly; Skip-gram learns from repeated local prediction tasks.

Q2. Given the co-occurrence probabilities $P(\text{ice} \mid \text{solid}) = \text{high}$, $P(\text{steam} \mid \text{solid}) = \text{low}$, $P(\text{ice} \mid \text{gas}) = \text{low}$, $P(\text{steam} \mid \text{gas}) = \text{high}$, explain how GloVe uses the ratio $P(\text{ice}|\text{context}) / P(\text{steam}|\text{context})$ to learn that "solid" and "gas" relate differently to "ice" and "steam".

Answer: For context word "solid": $\text{ratio} = P(\text{ice}|\text{solid}) / P(\text{steam}|\text{solid}) = \text{high} / \text{low} \rightarrow$ a large ratio, meaning "solid" is much more associated with "ice" than with "steam".

For context word "gas": $\text{ratio} = P(\text{ice}|\text{gas}) / P(\text{steam}|\text{gas}) = \text{low} / \text{high} \rightarrow$ a small ratio, meaning "gas" is much more associated with "steam" than with "ice".

For a neutral context word like "water" (related to both), the ratio would be close to 1.

GloVe's training objective fits vectors so that differences in dot products approximate the log of these ratios — so vectors for "solid"/"ice" and "gas"/"steam" end up capturing exactly this pattern, encoding meaning from the relative co-occurrence statistics rather than raw counts alone.

Q3. A researcher wants embeddings that support analogies like "king – man + woman = queen", using a huge web-scale corpus, without training a model from scratch. Recommend an approach involving GloVe and justify it.

Answer: Recommendation: Use pretrained GloVe vectors (e.g. trained on Common Crawl or Wikipedia + Gigaword).

Justification: Because GloVe is trained on aggregated global co-occurrence statistics from massive corpora, its published pretrained vectors already encode rich linear substructures — relationships like gender, capital-city-of, or tense — that make vector arithmetic (king – man + woman $\approx$ queen) work well. Using a pretrained model avoids the cost of collecting and training on billions of tokens while still benefiting from that scale.

6. Word Embedding (General)

Word embeddings map words to dense, low-dimensional vectors learned so that words with similar meaning or usage have similar vectors.

Q1. A chatbot needs to recognize that "purchase" and "buy" mean something similar, but a rule-based keyword-matching system misses this connection. How would replacing keyword matching with word embeddings fix this, and what property of embeddings enables it?

Answer: Keyword matching only recognizes exact string matches, so a query using "purchase" fails to match a rule written for "buy".

Word embeddings are learned from the distributional hypothesis — words that occur in similar contexts get similar vectors. Since "purchase" and "buy" tend to appear in similar sentences, their embedding vectors end up close together in vector space, giving a high cosine similarity. The chatbot can then match intent by vector similarity instead of exact words, correctly treating "purchase" and "buy" as (near) synonyms.

Q2. An e-commerce recommendation engine wants to represent product titles as vectors to find similar products, over a vocabulary of 50,000 words. Compare one-hot encoding versus dense (300-dimensional) word embeddings for this task.

Answer: One-hot encoding: each word is a 50,000-dimensional sparse vector with a single 1. Vectors are orthogonal, so there is no built-in notion of similarity between related words (e.g. "shirt" vs. "t-shirt"), and storing/comparing such high-dimensional sparse vectors at scale is memory- and compute-heavy.

Dense embeddings: each word is a compact 300-dimensional vector — roughly 166× smaller — that captures semantic relationships learned from data. Similar products (e.g. titles containing "sneakers" and "trainers") end up with vectors close together, so similarity search and clustering work meaningfully, unlike with one-hot vectors.

Conclusion: dense embeddings are both far more compact and far more useful for a similarity-based recommendation task.

Q3. Using trained word embeddings, explain the vector arithmetic behind the analogy "Paris – France + Italy = ?" and state the expected result.

Answer: $\text{vector}(\text{"Paris"}) - \text{vector}(\text{"France"})$ isolates a direction in the embedding space that roughly encodes the relationship "is the capital of" (subtracting the country from its capital city).

Adding $\text{vector}(\text{"Italy"})$ to that direction moves to "the capital of Italy": $\text{vector}(\text{"Paris"}) - \text{vector}(\text{"France"}) + \text{vector}(\text{"Italy"}) \approx \text{vector}(\text{"Rome"})$.

Expected result: Rome.

This works because good word embeddings encode relational structure (like capital-of, gender, or tense) as consistent linear directions in vector space, so the same offset applied to a different starting word lands near the analogous word.